Eleanor Query Language (EQL) — Specification

Status: Draft v0.1

Scope: normative specification of the query language; non-normative examples are marked as such.

1. Purpose

EQL is a declarative query language over a tree of Python dataclasses rooted at `Order`. It describes, without transforming, a stream of records to be extracted from that tree. EQL is consumer-agnostic: CSV writing, Postgres projection, navigator feedback, and assessor logic are all valid consumers of the same query.

EQL is strictly descriptive. It has no arithmetic, no comparisons, no function calls, no side effects. All transformation of extracted values is the consumer's responsibility.

2. Conformance terms

The key words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, **MAY** have the meanings given in RFC 2119. "Leaf" and "container" are defined in §3. "Alias" and "ambient scope" are defined in §5 and §7.

3. Object model

3.1 Reflection requirements

The object tree operated on by EQL **MUST** consist of Python dataclasses (`@dataclass`) with PEP 526 annotations. All annotations **MUST** be resolvable via `typing.get_type_hints()` — forward references **MUST** resolve in the module's globals.

Types not declared as dataclasses are permitted only as leaf values (§3.2) or as elements of non-traversed containers.

3.2 Field classification

Each annotated field of a dataclass is classified using its declared type after unwrapping `Optional[X]` / `X | None`:

- **Container field**: the unwrapped type is (a) a dataclass, (b) `list[X]`, or (c) `dict[K, V]`. The element type of a container is classified recursively.
- **Leaf field**: any annotated field that is not a container field.

EQL treats union types other than `T | None` as opaque leaves. If a field is declared `A | B` where both `A` and `B` are dataclasses, EQL **MUST NOT** navigate into it; it is reported as a leaf.

3.3 Leaf value handling

Leaf values are returned to consumers as their native Python objects. EQL assigns no special meaning to `datetime`, `bytes`, `Exception`, `None`, enums, or any other leaf type. All rendering is the consumer's concern.

`None` is a legal leaf value and **MUST NOT** be conflated with a missing path (§11).

Note: fields that were formerly top-level scalars on `ESPoint` but were moved to `custom_properties` (e.g. `solution_volume`, `charge_discrepancy`) are not accessible as direct path segments; they reside inside the `custom_properties` dict.

4. Grammar

```
Query         ::= "row_scope"   ":" RowScope
                  "columns"     ":" ColumnList
                  [ "on_missing" ":" MissingPolicy ]
                  [ "version"    ":" Integer ]

RowScope      ::= Identifier | Path                  ; §6

Path          ::= Segment ( "." Segment )* [ "." Meta ]
Segment       ::= Identifier Filter*
Meta          ::= "@" Identifier
Filter        ::= "[" FilterBody "]"
FilterBody    ::= "*" | Predicate ( "," Predicate )*
Predicate     ::= Identifier "=" Value
Value         ::= Unquoted | QuotedString
Unquoted      ::= UnquotedChar+
UnquotedChar  ::= any Unicode character except WS, "=", ",", "]", "\""
QuotedString  ::= "\"" ( NonQuote | EscapeSeq )* "\""
EscapeSeq     ::= "\\\"" | "\\\\"
Identifier    ::= [A-Za-z_] [A-Za-z0-9_]*

ColumnList    ::= list of ColumnEntry
ColumnEntry   ::= Path                               ; §8.1
                | StructuredColumn                   ; §8.2
                | SplatDirective                     ; §9
                | PresetDirective                    ; §10

MissingPolicy ::= "blank" | "null" | "error"
```

Filters on a single segment apply left-to-right. Multi-predicate filters **MAY** be spelled either way: `foo[a=1, b=2]` is equivalent to `foo[a=1][b=2]`.

`[*]` denotes iteration over a container. `[attr=value]` denotes a lookup filter; at runtime it **MUST** match zero or one element (see §11, §15).

`@<name>` is a *meta-accessor*: a closed-set, EQL-provided pseudo-segment that produces metadata about the path's iter-bound prefix rather than reading a real attribute. Meta-accessors are restricted to the terminal position of a column path; their semantics and the canonical names are defined in §7.1.

5. Aliases

5.1 Positional, not nominal

Every scope in a query carries an **alias** derived from the attribute path that reached it, never from the type name. Two distinct types may share a class name (e.g. `variable_space.Point` and `equilibrium_space.Point`); EQL distinguishes them by the attribute path through which they are reached.

5.2 Default aliases

- The root of every query has the alias `order`.
- For a segment `foo` whose declared type (after `Optional` unwrapping) is `list[T]` or `dict[K, T]`, each iterated element's alias is the **English singular** of `foo` (`vs_points → vs_point`, `elements → element`, `end_members → end_member`). If `foo` has no standard singularization under the rules given in §5.4, the alias is `foo` unchanged.
- For a segment `foo` whose declared type is a dataclass (not a collection), the alias is `foo`.
- The innermost scope of a query is additionally addressable as `self`.

5.3 Alias short-forms

A closed, code-defined table maps default aliases to shorter synonyms. Both the default and the short form are accepted wherever an alias is used.

Current entries:

default alias	short form
`vs_point`	`vs`
`es_point`	`es`

A short-form **MUST NOT** collide with any default alias. This table is not user-extensible; it is modified only alongside changes to this specification.

5.4 Singularization rules

EQL maintains a closed, code-defined table of irregular plural suffixes whose singular form is not produced by the algorithmic rules below. If an attribute name's lowercase form ends with one of the plural suffixes in this table, the matched suffix is replaced with the corresponding singular suffix and the algorithmic rules are not consulted. Suffix matching is case-insensitive; the surrounding prefix retains its original case.

Current entries:

plural suffix	singular suffix
`species`	`species`
`axes`	`axis`

This table is not user-extensible; it is modified only alongside changes to this specification.

Examples: `species → species`, `aqueous_species → aqueous_species`, `axes → axis`, `coordinate_axes → coordinate_axis`.

If no irregular suffix matches, EQL applies the following rules in order to derive the singular of an attribute name:

1. `…ies` ⟶ `…y` (e.g. `entries → entry`).
2. `…ses`, `…xes`, `…zes`, `…ches`, `…shes` ⟶ strip `es` (e.g. `suffixes → suffix`).
3. `…men` ⟶ `…man` (e.g. `end_members` — rule does not apply; falls through to rule 4).
4. Trailing `s` (not `ss`) ⟶ strip `s` (e.g. `vs_points → vs_point`).
5. Otherwise, the alias is the attribute name unchanged.

When the identity fallback in rule 5 (or an irregular form that resolves to itself) would create a collision with another ambient alias, the user **MUST** disambiguate via an explicit path and/or rely on the short-form table.

5.5 Alias uniqueness

Within a single compiled query, every ambient scope **MUST** have a unique alias. If reflection produces a collision within a single `row_scope`, the query is rejected at load time with `AliasCollision`, naming both paths.

6. `row_scope` resolution

`row_scope` is either a shortname or a full path.

6.1 Shortname

A single identifier. EQL enumerates every attribute path from `Order` that reaches a scope whose alias (default or short-form) equals the identifier:

- Exactly one candidate ⟶ `row_scope` expands to that path.
- Zero candidates ⟶ `UnknownRowScope`.
- Two or more candidates ⟶ `AmbiguousRowScope`, with candidate paths listed in the error.

6.2 Full path

Taken literally. Every segment **MUST** correspond to a real attribute of its parent type; every filter predicate **MUST** name a real field of the element type.

6.3 Terminal requirement

The terminal segment of a valid `row_scope` **MUST** evaluate to one of:

- A single dataclass node (no trailing `[*]`) ⟶ the query emits exactly one row per source tree.
- An iterative container segment (trailing `[*]`) ⟶ the query emits zero or more rows per iteration of the container.

A `row_scope` terminating at a leaf attribute, or at a bare `list[T]` / `dict[K, V]` without `[*]`, is rejected with `InvalidRowScope`.

7. Ambient scope table

Given a validated `row_scope`, the ambient scope table maps aliases to nodes in the current row:

- `order` ⟶ the root `Order`.
- For every non-filtering segment on the path, the segment's alias ⟶ the node at that position.
- Iterative `[*]` segments bind the currently iterated element to the corresponding singularized alias.
- The innermost alias is also bound to `self`.

Filtering `[attr=value]` segments do not introduce a new alias; they constrain the element reached via the surrounding attribute.

Columns **MUST** reference only aliases present in this table; references to unknown aliases are rejected at load time with `UnknownScope`.

7.1 Iter-bound aliases and meta-accessors

An alias bound by an iterative `[*]` segment is an *iter-bound alias*. For each iter-bound alias `<alias>`, EQL exposes a closed set of meta-accessors as terminal-only path extensions:

- `<alias>.@index` — the 0-based position of the currently iterated element within its parent container, typed `int`. Defined whenever `<alias>` is bound.
- `<alias>.@key` — for `dict[K, V]` iter scopes only, the dict key at the current iteration, typed `K`. Defined whenever `<alias>` is bound to a dict iteration; rejected at load time on list-iter aliases.

Meta-accessors are produced by EQL rather than read from the data tree; they never miss (§11.1) and carry no `on_missing` policy. Match-bound aliases — those reached only through an `[attr=value]`-only segment, with no enclosing `[*]` — do not expose meta-accessors. The set of canonical meta-accessor names is closed: unknown `@<name>` references and meta-accessors anchored on a non-iter-bound alias are rejected at load time with `InvalidMetaAccessor` (§15.1). New canonical meta-accessors are added only by modifying this specification.

8. Column specification

A column path **MUST NOT** contain an iter filter (`[*]`). Iteration over a container, and the resulting fan-out of rows, is the exclusive responsibility of `row_scope` (§6, §11.1). Match filters (`[attr=value]`) are permitted in column paths; per §15.2 they match exactly zero or one element at runtime, so a column always produces exactly one value per row (a leaf, or a container terminal under the consumer opt-in described in §13.1 step 6).

A column path containing an iter filter is rejected at load time with `InvalidFilter`.

A column path **MAY** terminate in a meta-accessor (§7.1); the column's value is then produced by EQL from the iter binding rather than read from the data tree. Meta-accessors are leaves and are unaffected by the consumer container-terminal opt-in.

8.1 Path string

A bare string is a path whose column name is derived per §8.5.

```
- es.ph
- es.aqueous_species[name=Ca+2].log_molality
```

8.2 Structured column record

A mapping whose keys are drawn from the following closed set:

- `path` (required, string) — the path to evaluate.
- `name` (optional, string) — explicit column name; overrides §8.5.
- `on_missing` (optional, `MissingPolicy`) — per-column override of the file-level policy.
- `default` (optional, any leaf value) — substitute used when `on_missing == null`.
- `meta` (optional, mapping) — consumer-facing annotations (§14). Keys **MUST** be strings; values are opaque. EQL **MUST NOT** interpret `meta`; it is carried through compilation unchanged and exposed on the compiled column for the consumer.

Any other top-level key on a structured column record is rejected at load time with `ParseError`. This keeps field typos (e.g. `on_mising`) from being silently accepted; consumer-defined keys **MUST** be nested under `meta` rather than placed at the top level.

```yaml
- path: es.aqueous_species[name=Ca+2].log_molality
  name: log_m_Ca
  on_missing: null
  default: null
  meta:
    greater_than: -6
```

8.3 Splat directive

See §9.

8.4 Preset directive

See §10.

8.5 Implicit column naming

For a path whose terminal segment is `foo` (after any filters), the default column name is `foo`. For a path whose terminal segment is a meta-accessor `@<name>`, the default column name is `<name>` (without the leading `@`).

If two or more columns in the compiled query would share the same default name, each colliding column is renamed to `<alias>_<name>`, where `<alias>` is the alias of the outermost scope on that column's path. Prefixing applies only to the colliding columns; other columns retain their bare names.

If prefixing does not resolve the collision (two paths agree after alias prefixing), the query is rejected with `ColumnNameCollision`; the user **MUST** supply explicit `name` values.

Numeric `_N` suffixes are never introduced by EQL.

9. Splat directive

```
- splat: <alias>
  [ exclude: [<field>, ...] ]
  [ include: [<field>, ...] ]     ; mutually exclusive with exclude
  [ prefix:  <string> ]
  [ on_missing: <MissingPolicy> ]
```

Resolution at compile time:

1. Look up `<alias>` in the ambient scope table to obtain its type `T`.
2. Enumerate the leaf fields of `T` (§3.2).
3. Filter by `include` or `exclude` if present. Unknown field names are a load-time error (`SplatUnknownField`).
4. Emit one column per remaining field: `path = <alias>.<field>`, `name = <prefix><field>`.

Splat **MUST NOT** recurse into containers. To project a nested container scope, write an additional splat directive or explicit columns.

Splat-produced columns inherit the directive's `on_missing`, if any, or the file-level default.

10. Preset directive

```
- preset: <name>
  [ <arg>: <value>, ... ]
```

A preset is a named function that desugars a single directive into a list of column entries (as if written by the user), spliced into the compiled column list at the directive's position. A preset function receives:

- the ambient scope table of the current query,
- the directive's arguments,

and returns the column entries it expands to. Preset expansion is recursive: a preset's output may itself contain bare-path, structured, splat, or preset entries, which are desugared in turn under the same bundle (§10.2).

A preset whose expansion requires an alias absent from the ambient scope table fails at load time with `PresetScopeMissing`, naming the missing alias.

10.1 Canonical bundle

The reference implementation ships a canonical bundle of presets (§10.3); a compile defaults to that bundle when none is supplied. The canonical bundle is closed: new canonical presets are added only by modifying this specification and the reference implementation together.

10.2 Per-compile bundles

A consumer of the reference implementation **MAY** supply a different bundle of presets at compile time, including the empty bundle. When a non-default bundle is in effect, that bundle (and not the canonical one) determines which preset names resolve and which raise `UnknownPreset`.

Queries that rely on the canonical bundle are portable across consumers; queries that rely on a custom bundle are portable only across consumers configured with that bundle. This trade-off is explicit: it lets consumers ship deployment-specific column projections without forking the spec, while keeping the canonical set portable.

10.3 Canonical presets

The canonical bundle defines:

- **`run_metadata`** — emits leaf columns describing the source `Order`. Requires the ambient `order` alias (always present per §7). Takes no arguments. Emitted columns: `order.tags`, `order.name`, `order.creator`, `order.notes`, `order.eleanor_version`, `order.create_date`. The run's identifier is deliberately absent: it is assigned by the consumer persisting the run, in an id space of that consumer's choosing, so it is not a field of the object tree and not addressable by a path (§4). A consumer that needs it in its output emits it itself.
- **`es_scalars`** — emits one column per scalar leaf of the equilibrium-space row's `ESPoint`. Requires the ambient `es` alias. Optional `exclude: [<field>, ...]` removes named fields from the output; optional `include: [<field>, ...]` restricts the output to the named fields. `include` and `exclude` are mutually exclusive. Unknown field names raise `SplatUnknownField` against the `es` alias.
- **`aqueous_species_table`** — emits one column per `(name, field)` pair against `es.aqueous_species`. Requires the ambient `es` alias. Required `names: [<species_name>, ...]` and `fields: [<aqueous_species_field>, ...]`; both must be non-empty lists of strings. For each pair, emits a column with path `es.aqueous_species[name=<name>].<field>` and column name `<field>_<name>`. Unknown `fields` entries raise `ParseError`; the `names` list is not validated against any data-model table.

Malformed preset arguments (wrong types, missing required arguments, unknown extra arguments) raise `ParseError` at compile time.

11. Missing-value semantics

11.1 Definition of "miss"

A path evaluation **misses** when, during runtime walk:

- an intermediate value is `None` and the next segment would be applied to it, or
- a `[attr=value]` filter matches no element of its container.

A `[*]` filter that finds an empty container is not a miss; it yields zero rows at the `row_scope` level or zero elements at inner levels.

Meta-accessors (§7.1) never miss: their value is fully determined by the iter binding that scopes them, so the missing-value policy of §11.2 does not apply to a column whose terminal is a meta-accessor.

11.2 Policy

Per-column policy, or the file-level default in its absence (priority: column > directive > file-level > implicit default `blank`):

- `blank`: the column's value is `None`.
- `null`: the column's value is the column's `default` if set, else `None`.
- `error`: evaluation raises `PathMissError`, identifying row index, column name, and the first missing segment.

11.3 Explicit `None` vs. miss

An attribute whose value is legitimately `None` (e.g. `es.Point.log_xi` when the equilibrium space point represents the `eq3` stage, where ξ is undefined) is **not** a miss when it is the terminal of a path; the column's value is `None`. It **is** a miss when the path continues through it.

12. Filter and value coercion

12.1 List filters

For a segment whose unwrapped type is `list[T]`, a filter `[attr=value]`:

- requires that `T` declare an attribute `attr`;
- coerces the literal `value` to `attr`'s declared type per §12.3;
- selects the element(s) whose `attr` compares `==` to the coerced value.

12.2 Dict filters

For a segment whose unwrapped type is `dict[K, V]`, a filter:

- `[key=value]` selects the entry whose key compares `==` to the coerced value. The literal is coerced to `K` per §12.3. `key` is reserved and does not shadow any value-type attribute of the same name.
- `[<attr>=value]` where `attr` is a field of `V` selects entries whose value's `attr` matches, coerced to that field's type.
- `[*]` iterates the dict's values (not its keys) in insertion order.

12.3 Value coercion

Given a declared target type `T` and a parsed literal string `s`:

- `T` is `str` → the value is `s` unchanged.
- `T` is `int` → `int(s, base=10)`; failure is `InvalidFilterValue`.
- `T` is `float` → `float(s)`; failure is `InvalidFilterValue`.
- `T` is `bool` → `"true"` / `"false"` (case-insensitive) are accepted; anything else is `InvalidFilterValue`.
- `T` is an `enum.Enum` subclass → lookup by name, then by value; failure is `InvalidFilterValue`.
- Any other `T` → `InvalidFilterValue`. Expansion of coercion targets is a specification-level change.

Coercion is performed once at query compile time. Compile-time coercion failures are reported as `InvalidFilterValue` with the offending segment.

13. Compilation and evaluation

13.1 Compilation (load time)

1. Parse the query text.
2. Parse and resolve `row_scope` (§6).
3. Reflectively validate the `row_scope` path against the `Order` tree.
4. Build the ambient scope table (§7).
5. Desugar all column entries (paths, splat, presets) into a flat list of structured column specifications.
6. Validate every column: the path begins with an ambient alias; no segment contains an iter filter `[*]` (§8); every attribute segment exists; every filter predicate names a real field of the preceding type; every filter value coerces successfully; the terminal produces a leaf value (consumers that can accept containers **MAY** relax this check via a consumer flag); any meta-accessor terminal names a defined accessor (`@index`, or `@key` for dict-iter aliases) anchored on an iter-bound alias (§7.1).
7. Resolve column names (§8.5) and check for irreducible collisions.

A valid query compiles to an opaque `CompiledQuery`. All errors detected after this point are runtime errors (§15.2).

13.2 Evaluation

```
evaluate(CompiledQuery, Order) -> Iterator[Row]
```

where `Row` is a mapping from column name to native Python value. Evaluation is:

- **Pure**: no I/O, no mutation of the input tree, no global state.
- **Deterministic**: identical inputs produce identical row streams.
- **Total** (with `on_missing != error`): every row is produced regardless of missing intermediates.

13.3 Iteration order

- `list[T]` iterates in storage order.
- `dict[K, V]` iterates in insertion order.
- Nested iterations compose left-to-right: the outer segment is the slowest index.

Consumers **MUST NOT** assume any other ordering, and producers **MUST NOT** depend on iteration order for semantic correctness.

13.4 Memory model

Evaluation **MUST** be streaming: at any point in time, only the current row's computed values and the ancestor-node pointers along the active `row_scope` walk are materialized. Consumers that buffer rows do so at their own cost.

14. Consumer contract

- Consumers receive the compiled column list and a row iterator.
- Consumers are responsible for rendering leaf values (formatting, null representation, byte handling, type mapping, etc.).
- Consumers **MAY** carry per-column render metadata out-of-band of EQL (e.g. a CSV sink's `format` overlay, a Postgres sink's type-cast hints). Such metadata **MUST NOT** alter the values produced by evaluation; only how they are serialized.
- A structured column **MAY** carry inline consumer annotations in its `meta` mapping (§8.2). EQL treats `meta` as opaque: it neither validates nor acts on the contents, and passes the mapping through to the consumer on the compiled column unchanged. A consumer **MAY** interpret its own `meta` keys to drive behaviour beyond serialization — including post-extraction row filtering or aggregation (e.g. a `greater_than` annotation used to drop rows). Such consumer behaviour **MUST NOT** change the row stream produced by `evaluate` (§13.2); it applies only downstream of evaluation, in the consumer. Annotation semantics are entirely consumer-defined and are therefore not portable across consumers unless they share a convention.
- Consumers **MUST NOT** mutate rows or the source tree.
- Two consumers evaluating the same compiled query against the same `Order` **MUST** observe identical row streams.

15. Error catalog

15.1 Load-time errors

Error	Raised when
`ParseError`	The query text does not conform to the grammar (§4).
`UnknownRowScope`	The `row_scope` shortname resolves to zero candidate paths.
`AmbiguousRowScope`	The `row_scope` shortname resolves to more than one candidate path.
`InvalidRowScope`	The `row_scope` terminates at a leaf or a non-iterated container.
`InvalidPath`	A segment names an attribute that does not exist on its parent type.
`InvalidFilter`	A filter is not permitted in its context (e.g. `[*]` in a column path, predicate naming a non-existent field, filter on a non-iterable, filter on an alias head).
`InvalidFilterValue`	A filter literal cannot be coerced to the target attribute's declared type.
`UnknownScope`	A column references an alias not in the ambient scope table.
`AliasCollision`	Two ambient scopes in one query would share an alias.
`ColumnNameCollision`	Two columns would share a name even after alias prefixing (§8.5).
`SplatUnknownField`	A splat's `include` or `exclude` names a field not on the splatted type.
`PresetScopeMissing`	A preset requires an alias absent from the ambient scope table.

Error	Raised when
`UnknownPreset`	A preset directive names a preset that is not defined in the bundle in effect for the compile (§10).
`InvalidMetaAccessor`	A meta-accessor (§7.1) names an undefined accessor, is anchored on a non-iter-bound alias, or applies `@key` to a non-dict iter scope.

15.2 Runtime errors

Error	Raised when
`PathMissError`	`on_missing == error` and a path misses (§11.1) during evaluation.
`MultipleMatchError`	A `[attr=value]` filter matches more than one element (§4).

16. Versioning

A query **MAY** declare `version: <int>`. The absence of `version` is treated as the current major version.

Breaking changes — to the grammar, alias rules, column desugaring, missing-value semantics, or evaluation model — increment the major version. Additions that do not change the meaning of existing valid queries (new canonical presets, new coercion target types, new short-form aliases) are minor and do not require a version bump on the query side. Per §10.2, queries that rely on a non-default preset bundle are portable only across consumers configured with that bundle.

17. Examples (non-normative)

17.1 Flat ES-point CSV

```yaml
row_scope: es
on_missing: blank
columns:
  - order.name
  - {splat: vs, exclude: [scratch]}
  - {splat: es, exclude: [custom_properties]}
  - es.aqueous_species[name=Ca+2].log_molality
  - es.aqueous_species[name=Cl-].log_molality
  - es.pure_solids[name=Calcite].affinity
```

17.2 Order-only summary (single-row per Order)

```yaml
row_scope: order
columns:
  - {splat: order}
```

Evaluates to exactly one row per input `Order`.

17.3 Deep-nesting: one row per equilibrium end-member

```yaml
row_scope: es.solid_solutions[*].end_members[*]
columns:
  - order.name
  - vs.temperature
  - es.ph
  - solid_solution.name
  - {splat: self}
```

`self` aliases the `EndMember`; `solid_solution` is the enclosing `SolidSolution`.

17.4 Dict access

```yaml
row_scope: vs
columns:
  - order.name
  - vs.temperature
  - path: order.species[key=Ca+2].value
    name: Ca_target
    on_missing: null
    default: null
```

17.5 Preset use

```yaml
row_scope: es
columns:
  - {preset: run_metadata}
  - {preset: es_scalars, exclude: [charge_discrepancy]}
  - {preset: aqueous_species_table,
     names: [Ca+2, Na+, Cl-, HCO3-],
     fields: [log_molality, log_activity]}
```

A preset's arguments are defined by the preset; EQL itself only validates that the named preset exists and expands successfully.

17.6 Iter-position via meta-accessor

```yaml
row_scope: vs_points[*]
columns:
  - vs.@index
  - vs.temperature
  - vs.ph
```

Yields one row per variable-space point, with the `index` column carrying the 0-based position of the row's `vs_point` within `vs_points`. Note: `es.ph` is not in scope here because `row_scope`

iterates `vs_points`, not `es_points`; accessing equilibrium data requires a deeper scope such as `vs_points[*].es_points[*]`.

```
row_scope: order.species[*]
columns:
  - species.@key
  - species.@index
  - species.amount
```

For a `dict[K, V]` iter scope, `@key` exposes the dict key at the current iteration and `@index` exposes its 0-based insertion-order position. The iteration itself is performed by `row_scope`, not in column paths. (Hypothetical dict scope shown here for illustration; real Eleanor types may not have one.)

18. Open questions

The following are explicitly **unresolved** in this draft and **MUST** be settled before v1.0:

- The behaviour of `[attr=value]` against an attribute whose declared type is itself a container (e.g. `list[str]`). The current draft rejects this at load time via §12.
- Whether consumers that can accept container-typed columns (e.g. a JSON sink) need a formalized opt-in beyond the §13.1 step-6 relaxation.
- Exact spelling of `Unicode` whitespace in the `Unquoted` rule. The reference implementation is expected to use `str.isspace()`.
- Whether match-bound aliases (those reached only through `[attr=value]`) should also expose `@index` (and `@key` for dict matches), reflecting the position of the matched element. The current draft restricts meta-accessors to iter-bound aliases only.